LocalCloudConfigurator

System Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document describes implementation ownership, architecture, persistence, security and operational boundaries. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-LocalCloudConfigurator-java. 1 detailed Java SysDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-LocalCloudConfigurator-go.

LocalCloudConfigurator

LocalCloudConfigurator fills an Arrowhead 5.2.2 local cloud with the support systems and application systems an operator picks. The operator picks the systems in a web UI or through the API. The configurator turns the selection into an ordered deployment plan and submits that plan to [ColonyOS](https://colonyos.io) as a workflow. A ColonyOS executor on the cloud host carries out each step. The first target is the GoCloud (Arrowhead-test-cloud-go). The executor runs the cloud's own scripts, so the configurator follows the same steps as the cloud's Makefile.

Parts

Part	Where	Role
Catalog	catalogs/gocloud.json	The cloud's selectable systems, their Compose services, alternates (Rust/Java swaps), dependencies, identity provisioning and policy scripts
Planner	internal/planner	Turns a selection into steps (provision -> up -> policy, then stop, then smoke) in dependency order
API + UI	localcloudconfigurator serve (container in the ColonyOS stack)	GET /api/catalog, GET /api/status, POST /api/plan, POST /api/deployments, GET /api/deployments/{id}; UI at /
Cloud executor	localcloudconfigurator executor (on the cloud host)	ColonyOS executor of type arrowhead-cloud-executor; serves lcc.status and lcc.step
ColonyOS stack	deploy/colonyos/compose.yaml	TimescaleDB, Colonies server v1.9.10, the API

Rules the executor enforces

- The executor checks every step against its own catalog. It does not trust the arguments of a process.

- Only catalogued Compose services can be started or stopped. Core systems are never touched. - Only catalogued scripts in the cloud's scripts/ directory can run.

- Services start with docker compose up -d --wait --no-deps <service>. Dependencies are explicit plan steps. Compose therefore never starts a Go service while its Rust or Java alternate runs (see the GoCloud swap targets).

- When an alternate runs, the system counts as present. The configurator neither starts nor stops it; the cloud's swap-*/restore-* targets manage it.

- Step output goes to the ColonyOS process logs with credential-like values masked. Secrets are never printed.

- No secret rotation and no PKI changes. Identities are provisioned through the cloud's provision-*-identity.sh scripts, which log in as Sysop with sysop.crt.

LocalCloudConfigurator is an operations tool. It is not an Arrowhead system: it does not register in the ServiceRegistry and has no Arrowhead identity. The new-system checklist (AMQP ACL, sysMonitor policy, Orchestration grants) therefore does not apply.

Build and test

Deploying and the smoke test are described in DEPLOYMENT.md.

Adding a cloud or a system

Add a system as an entry in the catalog: id, kind (infrastructure/support/application), services, and optionally alternates, requires, provision and policies. To support another cloud, write a catalog with its own composeProject and executorType. Then run one executor per cloud host with -catalog and -cloud-dir.

Listening port

serve listens on 127.0.0.1:8095 by default. Override with -listen host:port, SERVER_PORT=N or -server.port N; the port overrides keep the -listen host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Go interface and design differences

The native Go LocalCloudConfigurator is an operator API and embedded UI using net/http and ColonyOS. It is not assumed equivalent to the Java Arrowhead support-system wrapper. internal/api/api.go registers GET /, GET /healthz, GET /api/catalog, GET /api/status, POST /api/plan, POST /api/deployments and GET /api/deployments/{id}. There is no GET /api/deployments collection route in this handler. API calls require Authorization: Bearer <operator-token>, checked in constant time; an empty configured token or mismatch gives 401 with {error:"operator token required"}. This is an operator bearer token, not an Arrowhead IDENTITY-TOKEN. The health endpoint returns text ok. Security headers include X-Content-Type-Options, X-Frame-Options and Content-Security-Policy. TLS termination and network exposure must follow this implementation's deployment configuration; no Java mutual-TLS parity is inferred. Planning rejects unknown JSON fields, limits request input to 64 KiB, returns 400 on decoding errors, 502 on status lookup failure and 422 on planner rejection. A successful plan gives 200. A deployment with no steps gives 200 and an empty id; otherwise successful workflow submission gives 202. Source snapshots of the API, planner, catalog and ColonyOS adapter are included for exact data types and failure behavior.

Detailed implementation evidence

0 native Go SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Rust implementation

No Rust implementation folder for LocalCloudConfigurator was found in the designated primary Rust tree on 2026-11-09. No executable, build command, endpoint or feature parity is asserted. A future port must define architecture, state migration, service registration, DTO validation, authorization and transport coverage before it can replace this system.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-LocalCloudConfigurator-java/documentation/v5.2.2/LocalCloudConfigurator-v5.2.2-SysDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

LocalCloudConfigurator

System Design Description

Abstract
This document describes the implementation of the LocalCloudConfigurator support system of the Eclipse Arrowhead 5.2.2 framework in `ah5-LocalCloudConfigurator-java` of the primary Java source tree (`Eclipse-Arrowhead-java`).

Contents

1 Overview **3**

2 Implementation **4**
2.1 Implementation language and tools 4
2.2 Functional properties implementation 4
2.3 Non functional properties implementation 6
2.4 Deployment 6
2.5 Verification 6

3 Services **7**

4 References **7**

5 Revision History **8**
5.1 Amendments 8
5.2 Quality Assurance 8

1 Overview

This document describes the LocalCloudConfigurator system implementation, which realizes the LocalCloud-Configurator SysD [1]. In Section 2, we describe implementation details of the system.

2 Implementation

The implementation is located in `ah5-LocalCloudConfigurator-java` (artifact `arrowhead-localcloudconfigurator`, `localcloudconfigurator.jar`). The same JAR runs as the Arrowhead system (`serve`, default), as the host worker (`worker`) and as the retained command-line tools (`keygen`, `register`, `executor`, `plan`).

2.1 Implementation language and tools

- Java 21, Spring Boot 3.4 (Web, Security, Data JPA, Quartz), Log4j2 with the JDBC log table, MariaDB driver, BouncyCastle (ColonyOS signatures).
- Eclipse Arrowhead common library as a private vendored source copy (`vendor/`).
- Storage: MySQL 8.4 (private schema `ah_localcloudconfigurator`); the host worker uses files.
- Host worker: non-web Spring application with the JDK `HttpsServer`.

2.2 Functional properties implementation

Package `eu.arrowhead.localcloudconfigurator`:

Component	Responsibility
`LocalCloudConfigurator Main`	Command dispatch: Arrowhead system, `worker` (non-web application), tools.
`LocalCloudConfigurator SystemInfo`	System `LocalCloudConfigurator`; services `localCloud Configuration` and `generalManagement`.
`Catalog`	Loads and validates the catalog (names, ownership of databases, scripts, dependency cycles, defaults); dependency closure, protected components, catalog hash.
`Planner`	Plans: defaults and selection with dependencies in order, provision, start, policy, explicit stops in reverse order, optional verification; plan hash.
`Runner`	Runs one step after checking it against the catalog (owner, kind, script, catalog hash, protected components, running dependents, database order); `docker compose` argument lists; output bounded and redacted.
`DirectBackend`	Worker execution: one deployment at a time on a single thread, results written atomically after every step, state-directory lock, interrupted deployments marked failed at start-up and never replayed.
`CloudColony`, `colony/`	Legacy ColonyOS backend: signed RPC, chained `lcc.step` workflows, `lcc.status`.
`service/Deployment Coordinator`	Status, plan and serialized apply with the plan-hash check and the active-deployment guard; shared by the API (JPA store) and the worker (file store).
`service/Configuration Service`	Saved designs with validation and revision checks.
`service/WorkerClient`, `tls/MutualTls`	API side of the worker connection: mutual TLS with peer-name pinning and the CRL.

Component	Responsibility
`service/BackendConfig`, `ColonyBackend`	Catalog bean and backend selection (`lcc.backend`).
`jpa/`	Entities `SavedConfiguration` (`configuration`), `Deployment`, `ActiveDeployment`, `Logs`; `JpaDeploymentStore`.
`api/http/`	`LocalCloudConfigurationAPI` (strict JSON, 64 KiB, native status codes and error bodies), `GeneralManagementAPI`, response headers.
`security/`	`ConfiguratorAccessFilter` and `ConfiguratorAuthorizationService`: Sysop, whitelist or ConsumerAuthorization grant per operation.
`worker/`	`WorkerApplication` (beans of the worker) and `WorkerServer` (loopback HTTPS, client authentication required).

2.2.1 Database

Table	Columns
`configuration`	`id` (UUID), `name`, `systems`, `removals`, `layout` (JSON text), `reapply`, `smoke`, `revision`, `updated_at`.
`deployment`	`id`, `plan`, `request` (JSON text), `state`, `result` (last reported state and steps), `submitted_at`, `updated_at`.
`active_deployment`	At most one row (`id` 1): `deployment_id` of the deployment that blocks new ones until the backend reports `success` or `failed`.
`logs`	Log4j2 JDBC appender (`generalManagement get-log`).

`deployment/database/init.sql` creates the schema; `least-privilege.sh` restricts the application account to SELECT, INSERT, UPDATE and DELETE. Hibernate validates the schema at start-up.

2.2.2 Host worker

The worker is started as `java -jar localcloudconfigurator.jar worker --lcc...` by a per-user launchd service installed with `scripts/install-direct.py` (RunAtLoad, KeepAlive). It does not register in the ServiceRegistry; it is the configurator's local agent. Properties:

Property	Meaning
`lcc.worker.address`, `lcc.worker.port`	127.0.0.1, 8493; a non-loopback address is refused at start-up.
`lcc.worker.ssl.key-store`, `trust-store`, `*-password`	Worker certificate `LocalCloudConfiguratorWorker` (PKCS12), cloud truststore; passwords from a 0600 config tree.
`lcc.worker.ssl.crl-file`	Cloud CRL for client certificates (optional, recommended).
`lcc.worker.allowed-clients`	`LocalCloudConfigurator`.
`lcc.worker.cloud-dir`, `lcc.worker.docker`	Cloud view (`compose.yaml`, `scripts`, `.env`), Docker CLI.

Property	Meaning
`lcc.worker.state`, `lcc.catalog.file`	File state (`active-deployment.json`, `deployments/`, `workflows/`), the worker's own catalog copy.

Worker interface (HTTPS, client certificate required, JSON as in the IDD [2]):

Method	Path	Behaviour
GET	`/healthz`	`{"status": "ok", "executionBackend": "direct"}`.
GET	`/api/status`	Running Compose services and component states.
POST	`/api/plan`	Plan for a request against the worker's catalog.
POST	`/api/deployments`	Reviewed selection with `expectedPlanHash`; the worker rebuilds the plan and refuses a different hash (409); 202 with a `local-<uuid>` identifier.
GET	`/api/deployments/{id}`	State and step results of a worker deployment.

2.3 Non functional properties implementation

2.3.1 Security

Mutual TLS on 8492 and to the worker (SysD, Section Security); outsourced authentication and the access filter for every operation; security profile guard; DML-only database account; read-only root filesystem.

2.3.2 Power management

None. The `sysMonitor power` observation always reports line supply.

2.3.3 Internal monitoring

Logs to the console, a rolling file and the `logs` table; the worker logs to the console (launchd log files). sysMonitor observations every 30 s.

2.3.4 Configuration

Spring properties (SysD), secrets from `configtree:/run/secrets/`.

2.4 Deployment

`compose.yaml` (project `ah5-localcloudconfigurator`) runs `localcloudconfigurator` (image `arrowhead-localcloudconfigurator-standalone:5.2.2`, UID 10001, read-only root filesystem, port 8492 on the host's loopback by default, `host.docker.internal` mapped to the host) and `localcloudconfigurator-db` (MySQL 8.4.11 with TLS, internal network). `scripts/deploy.sh` puts both services into a cloud's Compose file and installs the worker; it changes a deployed cloud and is run only by an operator.

2.5 Verification

`mvn clean verify`: catalog, planner and runner tests; direct backend (persistence, no replay, single worker); deployment coordinator (guard across restarts, stale hash); a full-application MockMvc test of every `/api` route with identity and grant checks, size limit and strict JSON; a plan-to-deployment run against a worker over mutual TLS and refusal of a missing, foreign, wrongly named or revoked certificate; the security profile test. A gated test validates the schema against a fresh MySQL 8.4.11 container and checks the DML-only grants.

3 Services

Services produced	SysD ref	SD ref	IDD ref
`localCloudConfiguration`	[1]	[3]	[2]
`sysMonitor`	[1]	[4]	[5]
`generalManagement`	[1]	–	–
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
systemDiscovery, serviceDiscovery	[6]	[7]	–
identity	[8]	[9]	–
authorization (verify)	[10]	–	–

4 References

[1] J. Delsing, “LocalCloudConfigurator – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-LocalCloudConfigurator-java/documentation/v5.2.2/LocalCloudConfigurator-v5.2.2-SysD.pdf.

[2] ——, “localCloudConfiguration HTTP/TLS/JSON – Interface Design Description (LocalCloudConfigurator),” Eclipse Arrowhead, IDD 5.2.2, 2026, localCloudConfigurator-v5.2.2-localCloudConfiguration-HTTP-IDD.pdf.

[3] ——, “localCloudConfiguration – Service Description (LocalCloudConfigurator),” Eclipse Arrowhead, SD 5.2.2, 2026, localCloudConfigurator-v5.2.2-localCloudConfiguration-SD.pdf.

[4] ——, “sysMonitor – Service Description (LocalCloudConfigurator),” Eclipse Arrowhead, SD 5.2.2, 2026, localCloudConfigurator-v5.2.2-sysMonitor-SD.pdf.

[5] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (LocalCloudConfigurator),” Eclipse Arrowhead, IDD 5.2.2, 2026, localCloudConfigurator-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[6] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[7] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

[8] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[9] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-SD.pdf.

[10] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-SysD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-09	5.2.2	Document created for the Java-Spring implementation: components, database, host worker (non-web Spring application, loopback 8493, mutual TLS, not registered), deployment and verification.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	